

COMICCRAFT – PROJECT DESIGN PHASE

AI Comic Story Creator using Gemini Models

1. Design Overview

The ComicCraft design defines an end-to-end web application consisting of three primary components: Frontend, Backend, and AI Integration. The frontend collects user inputs and presents the generated comic, the FastAPI backend manages processing and routing, and the AI services generate story content and illustrations.

2. System Architecture

The proposed architecture is divided into three layers:

Frontend: HTML, CSS, and Jinja2 templates for input and comic presentation.

Backend: FastAPI application responsible for routing, input processing, AI workflow coordination, layout building, and PDF export.

AI Integration: Google Gemini APIs for structured story generation and Hugging Face Diffusers with Stable Diffusion for comic illustrations.

3. Frontend Design

The frontend is designed around a simple flow from comic input to comic preview and export. The main templates are:

index.html: collects story prompt, character name, setting, tone, and art style.

comic_preview.html: displays generated panels with title, image, scene description, captions, and narration.

export_success.html: confirms successful comic export and provides a path back to creating another comic.

4. Backend Design

FastAPI provides the server-side application structure. Routes receive form or JSON input, invoke the appropriate AI functions, organize the generated content, and render the required Jinja2 templates. The main application workflow connects outline generation, story generation, image generation, layout construction, and PDF export.

5. AI Integration Design

The AI pipeline is designed as a sequence. Gemini Flash generates a structured five-panel outline. Gemini Pro expands the outline into detailed narration and character dialogue. Stable Diffusion generates an illustration for each panel from its image prompt. The backend then combines the generated text and images for preview and export.

6. Core Module Design

The project documentation defines the following functional modules:

gemini_flash.py – generate_outline(): generates the structured five-panel outline.

gemini_pro.py – generate_story(): generates narration and character dialogue.

image_generator.py – generate_image(): generates and saves panel illustrations.

layout_builder.py – build_comic_layout(): associates images with panel stories and creates the layout structure.

exporters.py – save_pdf(): creates the multi-page PDF.

7. Route Design

The main routes described in the project documentation are:

/ – loads the homepage.

/generate – handles form submission and the complete comic-generation workflow.

/generate-comic/json – accepts JSON input and returns comic layout data and the generated PDF path.

/test-image – directly tests image generation.

/export-success – displays export confirmation.

8. Data Flow Design

The designed data flow is: User Input → FastAPI Route → Gemini Flash Outline → Gemini Pro Story → Stable Diffusion Images → Comic Layout → Jinja2 Preview → PDF Export. This flow keeps the major processing stages modular while allowing the backend to coordinate the complete comic-generation process.

9. Comic Panel Design

Each generated panel is designed to contain a panel number/title, AI-generated illustration, scene description, caption or ambient information, and narration/dialogue. The preview presents the panels sequentially so the user can review the complete storyline before export.

10. PDF Export Design

The PDF export module uses the generated panel images and story content to create a multi-page comic. The documentation specifies FPDF for PDF generation and the static/exports folder for storing the generated PDF.

11. Project Folder and Environment Design

The project is designed to use a Python virtual environment and a modular application structure. Dependencies include FastAPI, Uvicorn, Jinja2, python-multipart, Google Generative AI, Diffusers, Transformers, FPDF, Pillow, and Accelerate. Sensitive API credentials are intended to be stored through environment variables or a .env file.

12. Interface Design Principles

The interface is designed to be clean, simple, and focused on storytelling. The documentation describes centered layouts, readable typography, organized input fields, consistent buttons, rounded panels, and responsive presentation to support an accessible comic-generation experience.

13. Error Handling and Reliability Design

The backend design includes exception handling for unexpected failures such as API errors and JSON decoding problems. FastAPI HTTPException handling is used for route-level errors so that failures can be handled without disrupting the overall application flow.

14. Deployment Design

ComicCraft is designed for local deployment using Uvicorn. After dependency installation and environment configuration, the application is launched with the documented FastAPI/Uvicorn command and accessed through the local application URL. FastAPI's interactive documentation is also available through the /docs endpoint.

15. Future Design Extensions

The modular design described in the source documentation provides a foundation for future enhancements such as user profiles, comic libraries, and multi-page story arcs.

16. Design Summary

The ComicCraft design provides a modular architecture that connects user-friendly web input, FastAPI backend processing, Gemini-based story generation, Stable Diffusion image generation, comic layout construction, preview rendering, and PDF export into one end-to-end workflow.

Reference: ComicCraft - AI Comic Story Creator using Gemini Models (provided project documentation).